

GenY Super-App

Architecture Document

Ride-hailing + food-delivery platform

NestJS backend • Flutter mobile app • React admin dashboard

How each part works, and how the whole system fits together

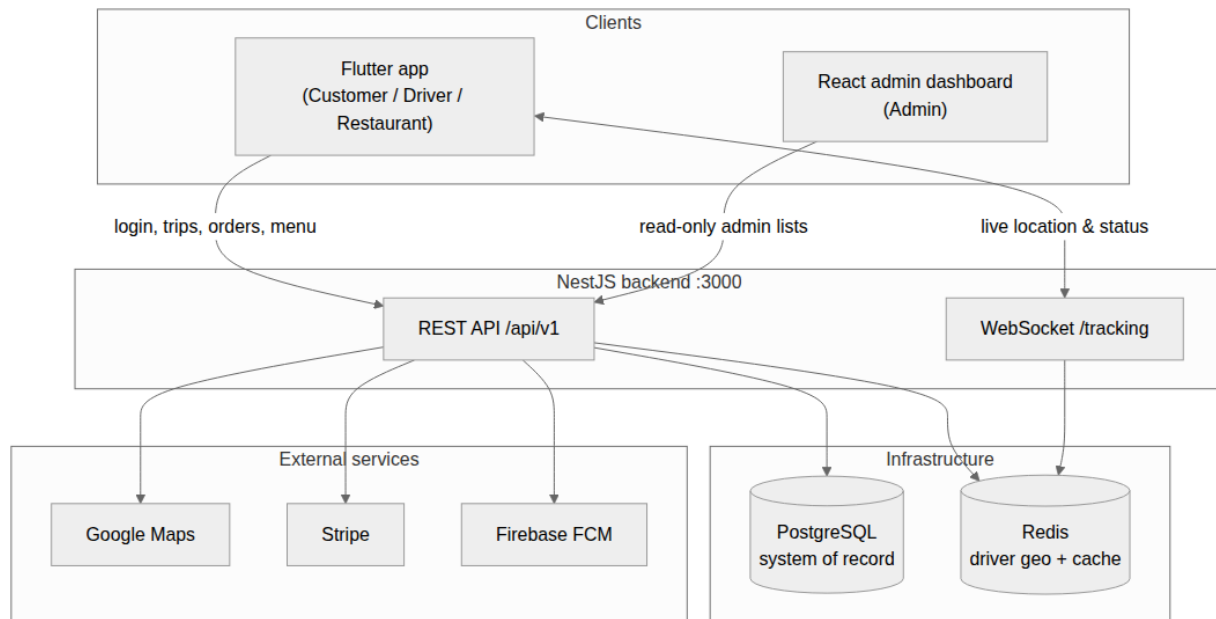
Contents

1. System overview

GenY is a "super-app": one platform that does ride-hailing and food delivery at once, positioned against Uber + Uber Eats and Careem + Talabat. It is built as a monorepo of three client surfaces sharing a single backend.

Component	Technology	Who uses it	Talks to backend via
Backend API	NestJS 10 (TypeScript)	—	—
Mobile app (passenger_app/)	Flutter	Customers, Drivers, Restaurants	REST + WebSocket
Admin dashboard (admin_dashboard/)	React 18 + Vite	Platform admins	REST

A single Flutter binary serves three of the four roles (Customer, Driver, Restaurant) and routes each user to the right experience after login. The React dashboard serves the fourth role (Admin). All four roles are defined once, on the backend, and every account has exactly one role.



Design principle — graceful degradation

Google Maps, Stripe, and Firebase are all optional. Each has a "mock mode": with no API key, Maps falls back to a haversine-distance calculation, Stripe returns simulated payment intents, and FCM becomes a no-op logger. This lets the entire product run end-to-end locally with zero external credentials.

Two data stores, two jobs

PostgreSQL is the system of record (users, trips, orders, payments — everything durable). Redis holds a geospatial index of online drivers for fast "nearest driver" queries, plus short-lived heartbeat keys. Losing Redis degrades matching but does not lose business data.

2. Technology stack

Concern	Backend	Mobile app	Admin dashboard
Language	TypeScript 5	Dart 3	TypeScript 5
Framework	NestJS 10	Flutter 3.13+	React 18
State / DI	Nest DI container	Riverpod	React hooks
Routing	Nest controllers	go_router	react-router-dom 6
Persistence	TypeORM → PostgreSQL 14	secure storage (tokens)	localStorage (token)
Cache / geo	Redis 4 (node-redis)	—	—
Auth	nestjs/jwt guard	JWT + refresh interceptor	JWT (Bearer)
Real-time	websockets + socket.io	web_socket_channel	—
Maps	Google Maps REST	google_maps_flutter	—
Payments	Stripe SDK	flutter_stripe	refund calls only
Push	Firebase Admin	firebase_messaging	—
Tests	Jest (153 unit + live E2E)	flutter_test	—

3. Repository layout

Each backend module is a self-contained bounded context: a module (wiring), a controller (HTTP routes), a service (business logic), DTOs (validated request shapes), and entities (TypeORM tables). Most also ship Jest unit tests.

```

test02/                                # repo root = the NestJS backend
├── src/
│   ├── main.ts                        # bootstrap: pipes, filters, interceptors, Swagger, WS
│   ├── app/                          # root module + config + common (guards/decorators/...)
│   ├── infra/                        # database, redis, google-maps, stripe, fcm
│   └── modules/                      # auth users drivers restaurants menu
├── scripts/                          # trips orders payments ratings notifications tracking
├── passenger_app/                   # smoke-test.mjs, e2e-full.mjs
└── admin_dashboard/                 # Flutter mobile app (3 roles in one codebase)
                                     # React admin dashboard

```

4. The backend: NestJS API

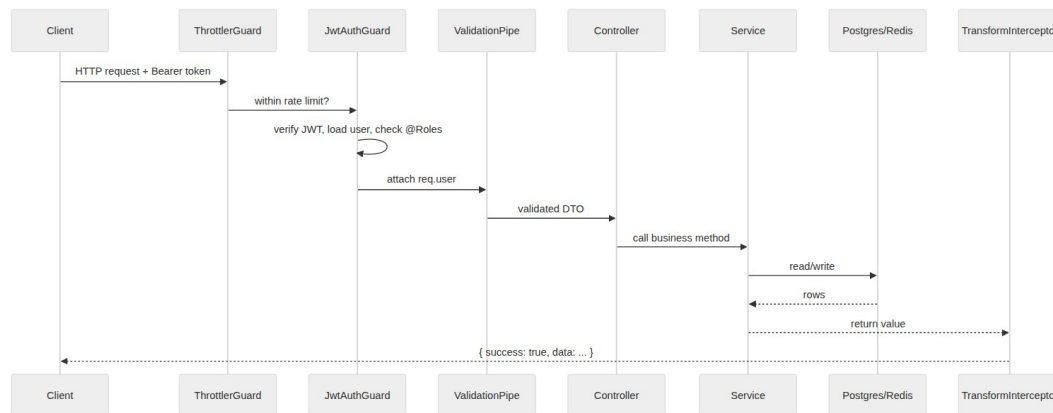
The backend is organized in three layers. `app/` is the composition root and everything cross-cutting (config, guards, the global response envelope, the global exception filter). `infra/` wraps external systems (Postgres, Redis, Google Maps, Stripe, Firebase). `modules/` is where the eleven domain modules and their business rules live.

Base URL: `http://localhost:3000/api/v1`. Interactive API docs (Swagger) at `/docs`.

4.1 Bootstrap and the request lifecycle

`main.ts` configures the app before it listens: a global `api/v1` prefix; a global `ValidationPipe` (whitelist + forbid unknown props + type coercion, so every body is validated against its DTO); the global exception filter; the `ClassSerializerInterceptor` (so password hashes never serialize) and `TransformInterceptor` (success envelope); the `socket.io WebSocket` adapter; and `Swagger`.

A typical authenticated request flows through the throttler, the JWT guard (verify token, load user, enforce roles), the validation pipe, the controller, the service, the database, and finally the transform interceptor which wraps the result:



If anything throws, the `AllExceptionsFilter` returns a uniform error instead: `{ success: false, statusCode, path, timestamp, message }`.

4.2 Cross-cutting layer

- `ThrottlerGuard` (global) — rate-limits every route to 120 requests per 60 seconds per client.
- `JwtAuthGuard` — the primary guard. Reads the Bearer token, verifies it, loads the user (rejecting inactive accounts), attaches `{ id, email, role }` to the request, and enforces any `@Roles(...)` on the route. It both authenticates and authorizes.
- `RolesGuard` — a lighter roles-only guard for contexts where the JWT was already verified upstream.
- `TransformInterceptor` — wraps every success as `{ success: true, data }`, giving all clients one envelope to unwrap.
- `AllExceptionsFilter` — catches everything; `HttpExceptions` keep their status, anything else becomes a logged 500.

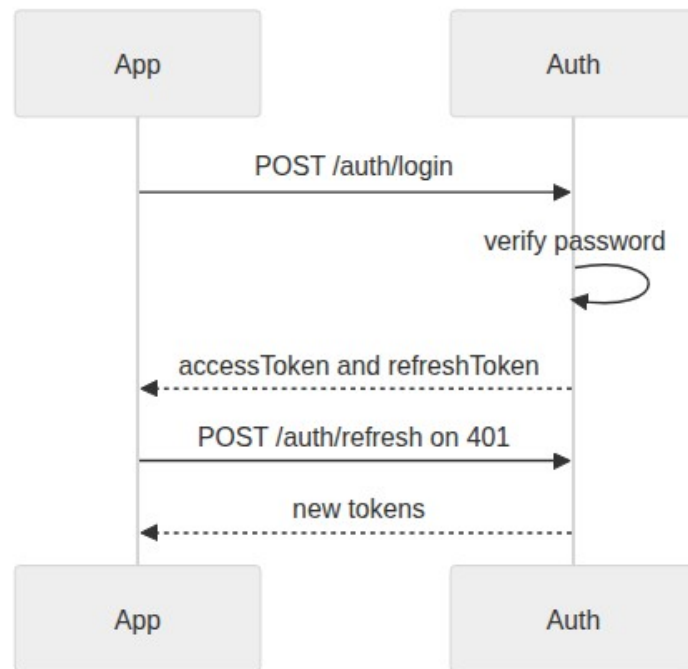
4.3 Configuration

Config is loaded through `nestjs/config` with typed loaders grouped by namespace (`jwt.*`, `googleMaps.*`, `stripe.*`, `database`, `app`). An `env-validation` schema checks required variables at startup so the app fails fast on misconfiguration. Blank or placeholder values for Maps / Stripe / Firebase trigger the mock-mode fallbacks.

4.4 Authentication & authorization model

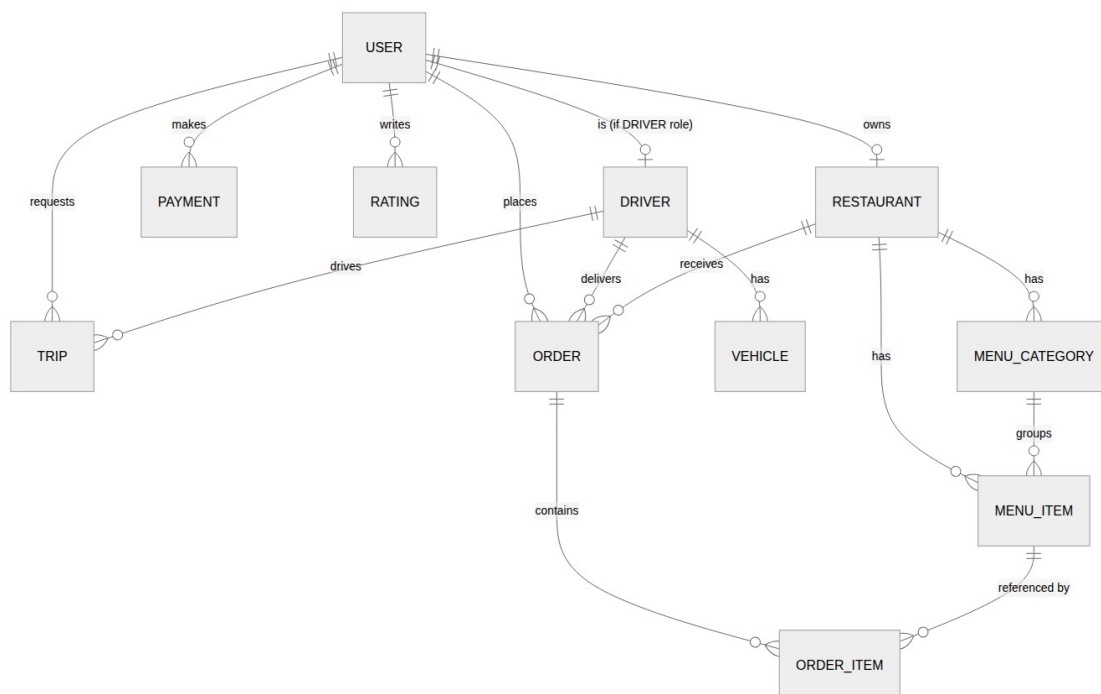
Four mutually-exclusive roles: CUSTOMER, DRIVER, RESTAURANT, ADMIN. One account = one role.

- Registration checks email/phone uniqueness, bcrypt-hashes the password, and refuses to create an ADMIN via public registration (a privilege-escalation guard). A DRIVER registration auto-creates a matching offline, unapproved driver profile.
- Login looks the user up by email or phone (identifier), verifies the password, and issues tokens.
- Tokens: a short-lived access token and a longer-lived refresh token, both signed over `{ sub, email, role }`.
- Refresh verifies the refresh token and re-issues a fresh pair. `/auth/me` returns the current user (password hash excluded).



4.5 The data model

Every table extends BaseEntity: a UUID primary key plus `created_at` / `updated_at` / `deleted_at` timestamps. The `deleted_at` column means soft deletes are used throughout — rows are hidden, not physically removed.



Entity highlights:

- User — email, phone (both unique), role, bcrypt password hash (excluded from serialization), FCM token, wallet balance, active flag, metadata.

- Driver — one-to-one with User; status (OFFLINE/ONLINE/ON_TRIP/ON_DELIVERY), type (RIDE/FOOD/BOTH), current position, aggregate rating, trip/delivery counts, approval flag. Has many Vehicles.
- Restaurant — owner, status (OPEN/CLOSED/BUSY), location, rating + count, delivery fee, minimum order, prep-time estimate, cuisine types, opening hours.
- MenuCategory / MenuItem — items carry price, availability, dietary flags, prep time, discount.
- Trip — customer, optional driver, the ride status machine, pickup/destination, distance/duration, fare estimate & final fare, surge, per-transition timestamps.
- Order + OrderItem — customer, restaurant, optional driver, the delivery status machine, the full money breakdown, delivery address, line items.
- Payment — purpose (TRIP/ORDER), reference id, provider intent id, amount, status. Rating — target (DRIVER/RESTAURANT), stars 1-5, comment, tags.

Schema management: the app runs with TypeORM synchronize:true for the MVP (tables auto-created from entities). Convenient locally but unsafe in production — migrations are the intended path.

4.6 Infrastructure services

- RedisService — set/get/del plus the geo helpers: addGeo (GEOADD), nearby (GEOSEARCH sorted by distance), removeGeo. These power matching.
- GoogleMapsService — wraps Distance Matrix, Directions, Geocoding. In mock mode it returns deterministic geometry from a haversine great-circle distance. Its estimateFare implements the pricing formula. Single source of both ETAs and fares.
- StripeService — PaymentIntents, refunds, webhook signature verification. In mock mode returns fake pi_mock_intents and refunds so payments are exercisable with no Stripe account.
- FcmService — Firebase Admin messaging; a no-op logger when Firebase is not configured.

4.7 Domain modules, function by function

auth

- Controller: POST /auth/register, POST /auth/login (200), POST /auth/refresh (200), GET /auth/me (guarded). Logic covered in 4.4.

users

- findAll (admin list), findOne, findByEmailOrPhone, update, deactivate (isActive=false), setFcmToken, updateWallet (atomic increment).
- Routes: GET/PATCH/DELETE /users/me, GET /users (admin), GET /users/:id.

drivers

- goOnline — requires approval; sets ONLINE, stamps lastSeenAt, adds the driver to the Redis geo index.
- goOffline — sets OFFLINE, deletes the ping heartbeat, removes the driver from the geo index.
- pingLocation — the 5-10s heartbeat: updates DB position, re-adds to the geo index, writes a 30s TTL liveness key.
- nearby — the map query: Redis geo search then load drivers; returns id, position, rating, type, distance.
- Vehicles: addVehicle (unique plate), listVehicles, verifyVehicle (ownership-checked).

restaurants

- create (owner = caller), findAll (public, active-only, filterable, ordered by rating), findOne, findByOwner, assertOwned (reused by menu), update, remove (soft delete), setStatus.

- Route ordering: GET /restaurants/me is declared before GET /restaurants/:id so the literal path is not captured as an id.

menu

- Category and item CRUD, each write guarded by restaurants.assertOwned so only the owner (or admin) can modify a menu. createItem verifies the category belongs to that restaurant. Reads are public and eager-load relations.

trips (ride-hailing)

- requestTrip — Maps distance/duration, fare estimate, persists a REQUESTED trip.
- acceptTrip — validates driver approved + online + trip still REQUESTED; assigns driver, ACCEPTED, driver → ON_TRIP, notifies customer (push + WS).
- updateStatus — only the assigned driver may set DRIVER_ARRIVING/ARRIVED/STARTED/COMPLETED; customer may cancel. On COMPLETED: recompute final fare, driver → ONLINE, totalTrips++, push notification. Every transition broadcasts trip:update.
- findNearestDriverForTrip — the ride-matching query.

orders (food delivery)

- create — validates restaurant open + items available + belong to restaurant; computes subtotal + distance-based delivery fee + 5% service fee + 5% tax + total; enforces minimum order; persists order and line items in one transaction.
- updateStatus — role-aware: customer may cancel before pickup; restaurant sets ACCEPTED/PREPARING/READY/REJECTED; on READY auto-assigns the nearest food driver and marks them ON_DELIVERY; the assigned driver sets PICKED_UP/ON_THE_WAY/DELIVERED (on DELIVERED: totalDeliveries++, driver → ONLINE). Each transition broadcasts order:update and notifies the customer.
- assignDriver — the food-matching query.

payments

- createIntent (Stripe intent + PENDING payment row, returns paymentId + client secret), listAll (admin), getOne, refund (full/partial). Webhook lives on a separate unguarded controller, authenticated by Stripe signature.

ratings

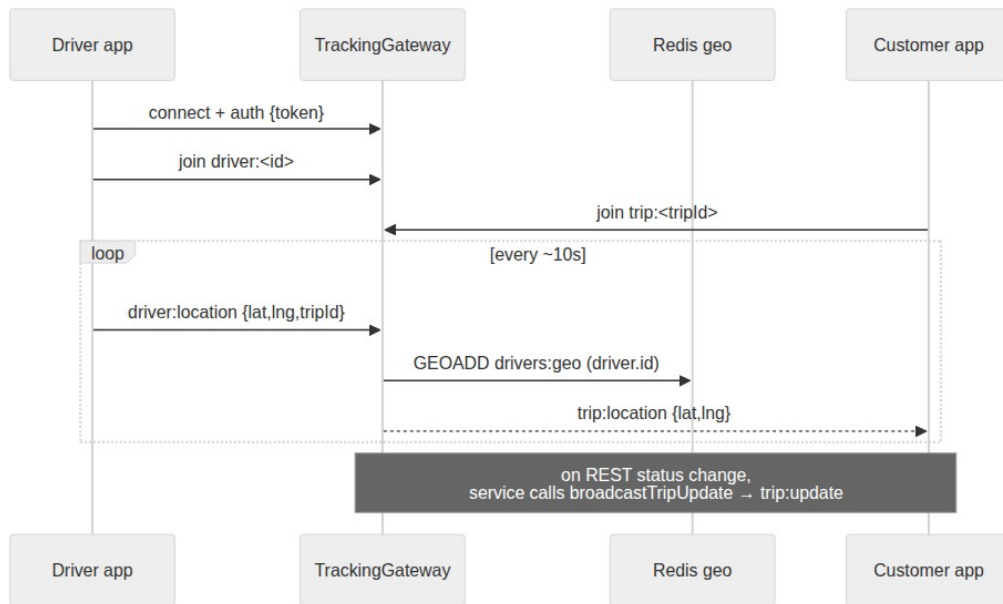
- create persists a 1-5 star rating then recomputes the target aggregate: Driver.rating, or Restaurant.rating + ratingCount, via AVG/COUNT. findByTarget / findByReviewer list ratings.

notifications

- sendToUser resolves the user FCM token and dispatches via FcmService (skipping gracefully if absent); sendToTopic broadcasts. The seam every module calls to push a notification.

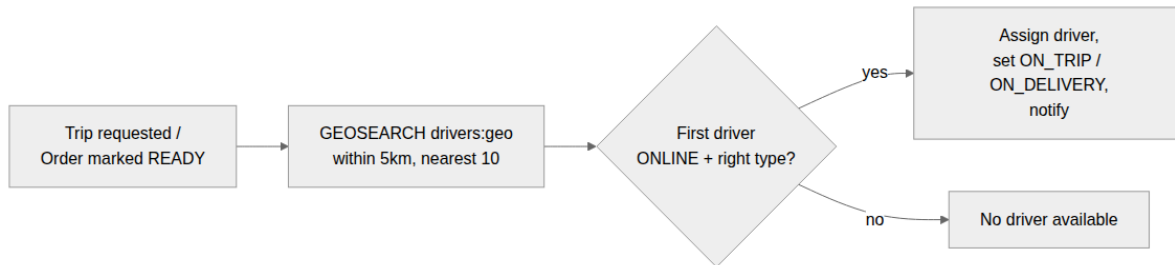
4.8 Real-time tracking gateway

TrackingGateway is a socket.io gateway on the tracking namespace. Rooms: trip:<id>, order:<id>, driver:<id>. Clients authenticate via the socket handshake (or a later auth message). A driver pushing driver:location has its position written to the Redis geo index under the same member id the REST ping uses — keeping one consistent index — and relayed to the relevant rooms. Services push status changes via broadcastTripUpdate / broadcastOrderUpdate / broadcastDriverOffer.



4.9 The driver-matching engine

Matching runs off the Redis geo index. For rides, a GEOSEARCH around the pickup (5 km, nearest 10) picks the first ONLINE driver of type RIDE or BOTH. For food, when an order hits READY, the same search around the restaurant picks the first ONLINE driver of type FOOD or BOTH, marks them ON_DELIVERY, and attaches them to the order.



The current engine is synchronous and single-shot — it offers to the single nearest eligible driver inline. The roadmap calls for an async engine (BullMQ) with cascade offers and accept timeouts.

4.10 The fare model

Fares use an Uber-style linear formula in `GoogleMapsService.estimateFare`:

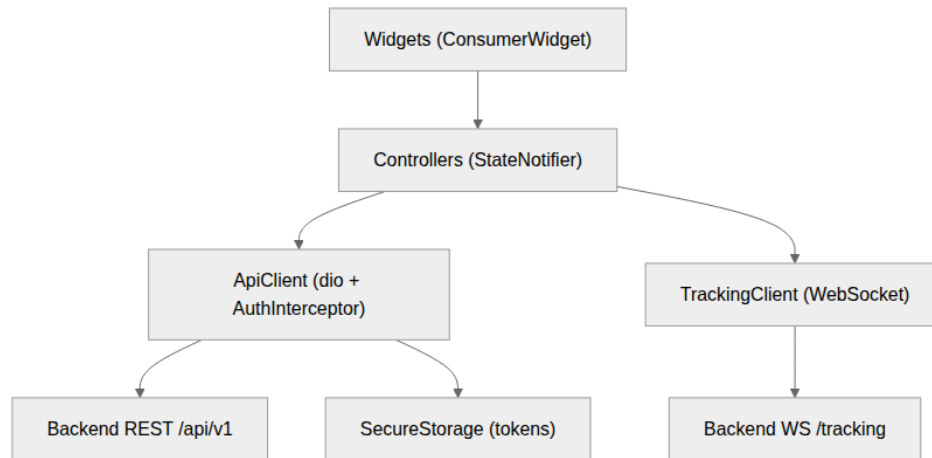
$$\begin{aligned} \text{fare} &= (\text{baseFare} + \text{perKm} \times \text{km} + \text{perMin} \times \text{min}) \times \text{surge} \\ &= (2.50 + 1.20 \times \text{km} + 0.25 \times \text{min}) \times 1.0 \end{aligned}$$

km and min come from Distance Matrix (or the haversine fallback in mock mode). Surge is currently hard-coded to 1.0; the intended production behavior derives it from the ratio of active requests to nearby online drivers. The estimate is stored at request time; the final fare is recomputed from actual distance on completion.

5. The mobile app: Flutter

`passenger_app/` is a single Flutter codebase that serves Customers, Drivers, and Restaurant partners, routing each to the right experience after login.

Architecture: layered and Riverpod-driven. UI widgets watch Riverpod providers → controllers (StateNotifiers) hold feature state → a typed ApiClient (over dio) and a TrackingClient (WebSocket) talk to the backend. Tokens live in flutter_secure_storage. A single go_router instance enforces auth and role gating.



Bootstrap & routing

main.dart initializes Firebase (optional), sets the Stripe key only if present, and runs inside a ProviderScope. Every route is wrapped in an FcmRegistrar that pushes the device FCM token to PATCH /users/me once authenticated. The router starts at /splash while the session is restored, forces unauthenticated users to /login, and sends authenticated users to their role home (/passenger, /driver, /restaurant, /admin). Role-locked shells bounce a user who tries to reach another role area.

Data layer

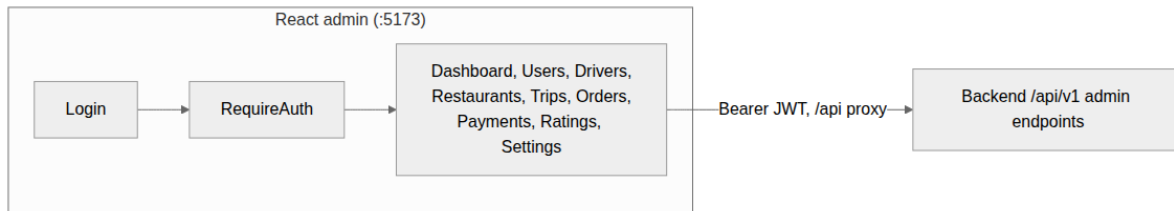
- ApiClient — a typed method per backend endpoint (auth, user, driver, restaurant/menu, trips, orders, payment intent, ratings). Centrally unwraps the { success, data } envelope. Its AuthInterceptor attaches the Bearer token and, on a 401, performs a single-flight token refresh (queuing concurrent requests, replaying after refresh, clearing storage if refresh fails).
- SecureStorage — namespaced geny.* keys for tokens, role, user identity, FCM token; cleared on logout.
- TrackingClient — connects the WebSocket, sends an auth frame, exposes a broadcast event stream, offers join(room) and sendDriverLocation(...), keeps a heartbeat, and auto-reconnects.

Feature controllers

- AuthController — login/register/logout/session-restore; every transition notifies the router.
- DriverHomeController — goOnline (API + socket + subscribe to offers + a 10s geolocation ping loop hitting both the REST ping and the WS location), goOffline, acceptTrip, updateTripStatus, advanceOrder (PICKED_UP → ON_THE_WAY → DELIVERED).
- FoodCartController — in-memory cart math (add/increment/decrement, subtotal, shaping items for POST /orders).
- RestaurantController — loads the partner restaurant, its incoming orders and menu; advances order status; full menu CRUD.
- PassengerHomePage — a Google Map with nearby-driver markers and the ride-request sheet (pickup/destination → fare estimate → place trip). historyStreamProvider backs the trips/orders/deliveries history tabs.

6. The admin dashboard: React

admin_dashboard/ is a React 18 + Vite SPA for the Admin role. In development Vite runs on :5173 and proxies /api → http://localhost:3000, so no CORS or base-URL config is needed.

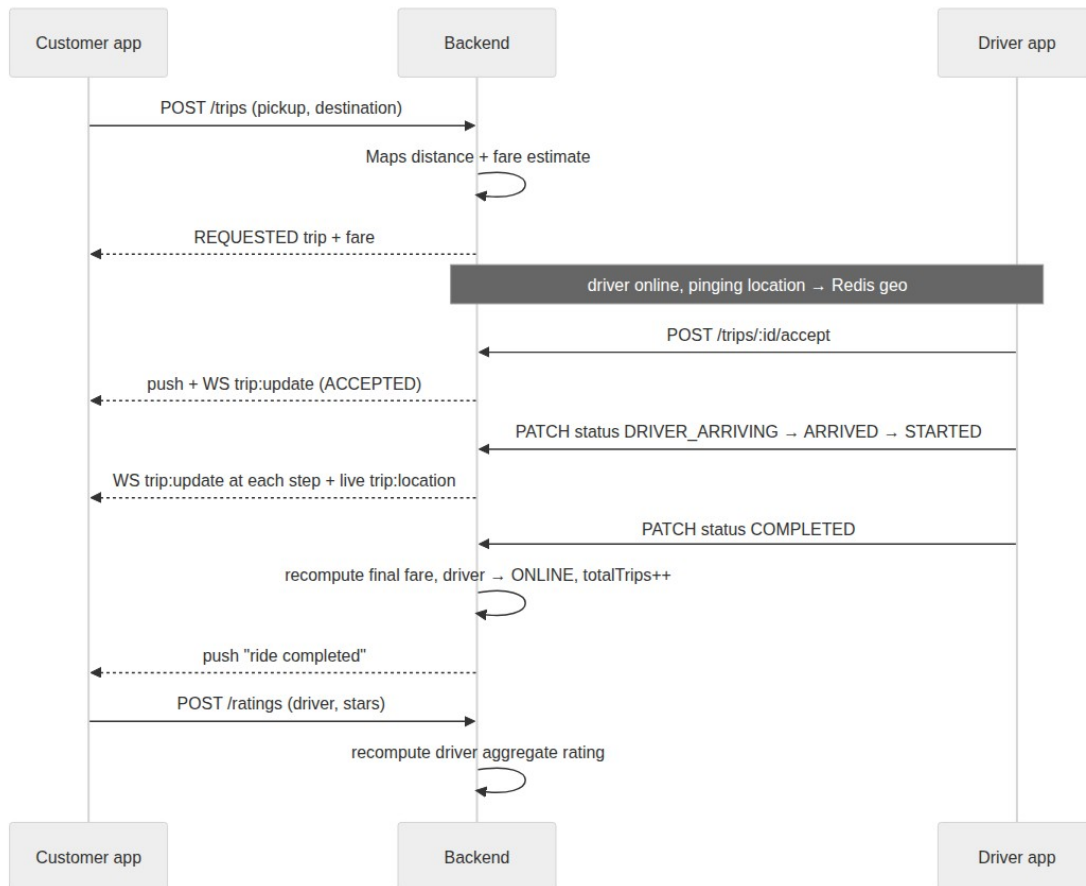


- Routing (App.tsx): a public /login, and everything else nested under a RequireAuth → AdminLayout boundary. Pages: Dashboard, Users, Drivers, Restaurants, Trips, Orders, Payments, Ratings, Settings.
- Auth (useAuth.ts + api.ts): login stores the JWT in localStorage under geny.admin.token and rejects any non-ADMIN account client-side; session is restored via GET /auth/me (ADMIN-gated). Every request attaches the Bearer token; a global 401 handler clears the token and redirects to /login.
- API client: listUsers, listDrivers, listRestaurants, updateRestaurant, deleteRestaurant, listTrips, listOrders, listPayments, refund, listRatings.
- Dashboard: there is no server-side aggregation endpoint — it fetches the list endpoints and computes KPIs in the browser (total users, driver count, active trips, active orders, gross revenue from delivered orders).

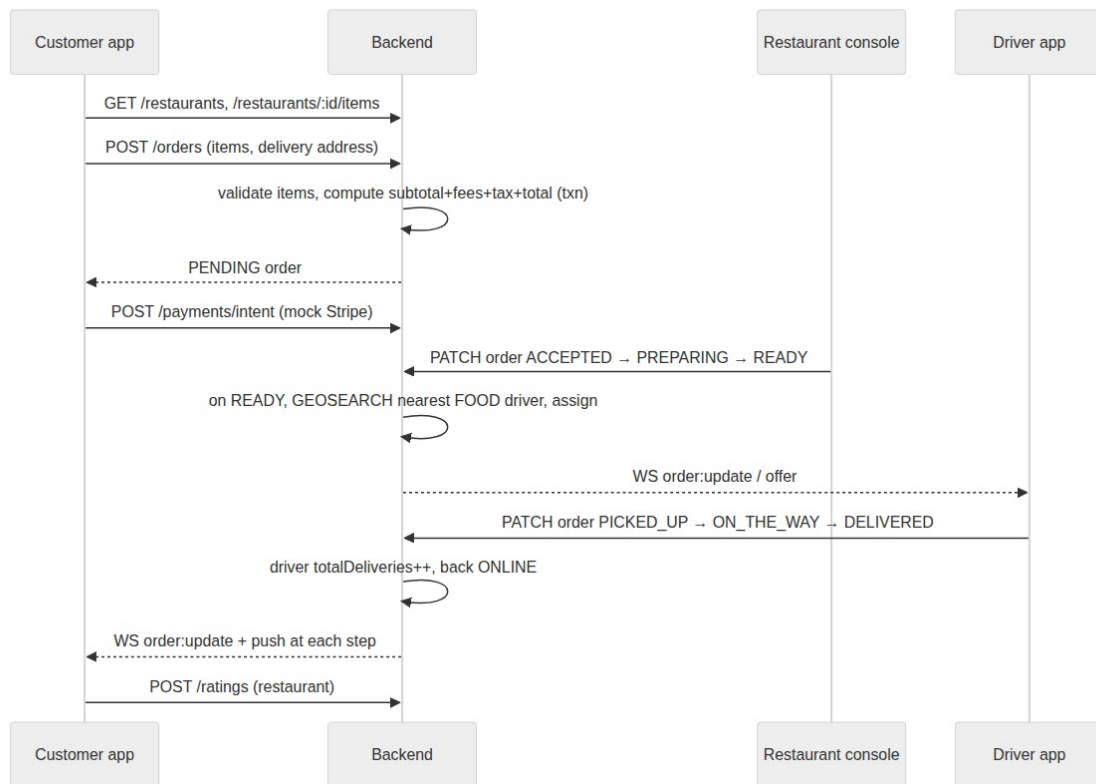
The dashboard is intentionally read-mostly: nearly every page is a GET list; the only writes are restaurant edit/delete and payment refund.

7. End-to-end flows across all apps

A ride, start to finish



A food order, start to finish



The admin dashboard observes all of this after the fact through the read-only list endpoints, computing platform KPIs from the raw lists.

8. Testing

- Unit tests (Jest) — 153 service specs across every module (npm test). They mock the database and exercise business logic in isolation.
- Live end-to-end suite — scripts/e2e-full.mjs (52 checks) drives real HTTP + WebSocket traffic against a running server with real Postgres + Redis, covering the full ride and delivery lifecycles, authorization negative cases, payments, ratings, and tracking. scripts/smoke-test.mjs is a lighter 13-check version.

Because the unit tests mock persistence, they do not catch integration issues (wiring, SQL, relation mapping, route ordering) — that is the job of the E2E suite. Both are green on the current main.

9. Running the system

```

# 1. Infrastructure (Postgres + Redis)
docker compose up -d

# 2. Backend
npm install
cp .env.example .env
npm run start:dev
npm run seed

# blank Maps/Stripe/Firebase keys = mock mode
# http://localhost:3000, Swagger at /docs
# 4 demo accounts (password: P@ssw0rd)

# 3. Mobile app
  
```

```
cd passenger_app && flutter pub get && flutter run

# 4. Admin dashboard
cd admin_dashboard && npm install && npm run dev    # :5173
```

Seed accounts (all password P@ssw0rd): admin@geny.app, customer@geny.app, driver@geny.app, burgers@geny.app (owns "GenY Burger House" and its menu).

10. Known issues & roadmap

Intentionally out of MVP scope (from the project roadmap): BullMQ async matching with cascade offers, multi-driver bidding, real surge pricing, scheduling / ride-pooling, S3/Cloudinary image uploads, driver KYC, multi-language/currency, Kubernetes manifests.

Notable current gaps to be aware of:

- Authorization returns 401 where 403 is more correct — the shared guard throws Unauthorized for a wrong-role (but authenticated) user.
- synchronize:true is enabled for the MVP; production should disable it and use migrations.
- Matching is synchronous and single-shot (no offer timeout / cascade); surge is hard-coded to 1.0.
- Flutter: the driver app does not join the trip:<id> room after accepting; the WebSocket auto-reconnect flag is not re-armed after a manual disconnect; the restaurant console polls rather than subscribing to live order pushes.
- Admin: KPIs are computed by fetching entire list endpoints and counting client-side (will not scale); the payments list endpoint is thin; a few dashboard deltas are placeholder strings.

Generated from a full read of the backend, Flutter app, and admin dashboard source.